Course code: CSE 404

Course Title: Artificial Intelligence and Expert Systems Lab

Lab Title: Knowledge-Based Recommender System for Movies

Lab Report: 01
Submission Date: 12-08-2025

Submitted By:	Submitted To:
Md Tausif Uddin	**Bidita Sarkar Diba**
ID: 22101112	Lecturer
Section**:** C-1	Department of Computer
Semester: 4-1	Science and Engineering
Session: Spring 2025	University of Asia Pacific

Problem Title

Movie Recommendation Knowledgebase: A Prolog Implementation for Personalized Movie Suggestions

Problem Description

This project implements a Prolog-based movie recommendation system that provides users with personalized movie suggestions based on their preferred genres, directors, and collaborative filtering from similar users. The system stores movie facts, user preferences, ratings, and rules to produce recommendations through multiple approaches such as genre matching, director matching, and user similarity analysis.

Main Features:

- **Multi-user support**: Stores preferences and ratings for multiple users such as User_A, User_B, and User_C.
- **Movie facts storage:** Maintains a list of movies with their genres and directors, e.g., *movie_1, genre_1, director_1*.
- **Preference-based recommendations:** Suggests movies based on a user's preferred genres and directors.
- **Meal tracking:** Monitors meal consumption and compares calories consumed versus calories burned.
- **Ratings tracking:** Stores ratings (1–5 scale) for each movie per user.
- **Collaborative filtering:** Identifies users with similar taste based on rating similarity and recommends movies accordingly.
- **Combination recommendation rule:** Provides suggestions from genre, director, or collaborative filtering in a unified query.

Tools and Languages Used

- **Programming Language:** Prolog
    - Prolog is used to create the movie knowledge base, define recommendation rules, and perform collaborative filtering.
- **Tools:**
    - SWI-Prolog for running and testing the knowledge base and rules.

Diagram/Figure:

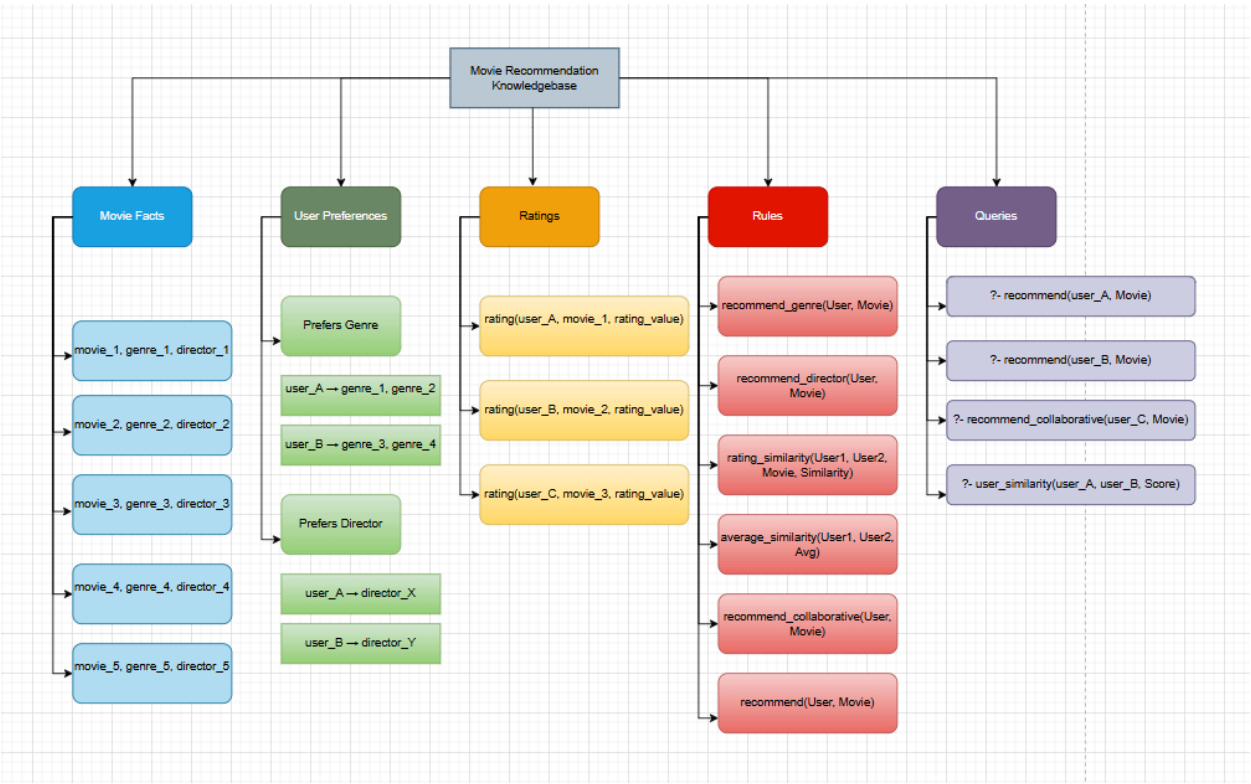

Fig: Knowledge base diagram of Movie Recommendation System

Explanation:

Static Facts:

Movies: Stores movie details such as:

- movie(movie_1, genre_1, director_1).
- movie(movie_2, genre_2, director_2).
- movie(movie_3, genre_3, director_3).

User Preferences:

Prefers Genre:

- prefers_genre(user_A, genre_1).
- prefers_genre(user_A, genre_2).

- prefers_genre(user_B, genre_3).
- prefers_genre(user_B, genre_4).

Prefers Director:

- prefers_director(user_A, director_X).
- prefers_director(user_B, director_Y).

Ratings:

- rating(user_A, movie_1, rating_value).
- rating(user_A, movie_2, rating_value).
- rating(user_B, movie_3, rating_value).
- rating(user_B, movie_4, rating_value).
- rating(user_C, movie_5, rating_value).

Utility Rules:

- `recommend_genre(User, Movie)` – Suggests movies based on genre preference.
- `recommend_director(User, Movie)` – Suggests movies based on director preference.
- `rating_similarity(User1, User2, Movie, Similarity)` – Calculates rating difference for a single movie.
- `average_similarity(User1, User2, Avg)` – Calculates average rating difference across shared movies.

High-Level Rules:

- `recommend_collaborative(User, Movie)` – Suggests movies liked by similar users (Avg difference ≤ 1.5 and rating ≥ 4).
- `recommend(User, Movie)` – Combines all recommendation strategies into a single query.

Sample Input/Output

1. **Recommend by Genre:**

- **Query & Output:**

```
?-
% c:/Users/USER/Desktop/prolog.pl compiled 0.00 sec, 35 clauses
?- recommend_genre(user_a, Movie).
Movie = blade_runner ;
Movie = inception ;
Movie = interstellar ;
Movie = godfather.
```

2. **Recommend by Director:**

- **Query & Output:**

```
?- recommend_director(user_a, Movie).
Movie = inception ;
Movie = interstellar ;
Movie = dark_knight.
```

3. **Rating similarity (works only if both rated the movie):**

- **Query & Output:**

```
?- rating_similarity(user_a, user_c, inception, Sim).
Sim = 1.
```

4. **Shared movies between users:**

- **Query & Output:**

```
?- shared_movies(user_a, user_c, Movies).
Movies = [inception, godfather].
```

5. **Average rating similarity:**

- **Query & Output:**

```
?- average_similarity(user_a, user_c, Avg).
Avg = 1.
```

6. **Collaborative filtering:**

- **Query & Output:**

```
?- recommend_collaborative(user_a, Movie).
Movie = pulp_fiction ;
Movie = inception ;
Movie = godfather ;
Movie = interstellar ;
Movie = dark_knight ;
Movie = inception.
```

7. **All recommendations (may repeat movies):**

- **Query & Output:**

```
?- recommend(user_b, Movie).
Movie = pulp_fiction ;
Movie = amelie ;
Movie = pulp_fiction ;
Movie = blade_runner ;
Movie = inception ;
Movie = godfather ;
Movie = interstellar ;
Movie = dark_knight ;
Movie = inception.
```

8. **All recommendations without duplicates:**

- **Query & Output:**

```
?- recommend_no_duplicates(user_b, Movie).
Movie = amelie ;
Movie = blade_runner ;
Movie = dark_knight ;
Movie = godfather ;
Movie = inception ;
Movie = interstellar ;
Movie = pulp_fiction.
```

Challenges:

- **Data scaling:** More movies and users will increase complexity and query time.
- **Collaborative filtering limits:** Works best when there is significant overlap in rated movies.
- **Preference conflicts:** A user may like multiple genres or directors, requiring careful handling to avoid irrelevant recommendations.
- **Accuracy dependence:** Incorrect or sparse ratings reduce the quality of recommendations.

Conclusion:

The Prolog-based movie recommendation system successfully integrates multiple recommendation strategies into a unified platform. It provides personalized suggestions based on genres, directors, and collaborative filtering, making it versatile for different user tastes. The modular structure of facts and rules allows easy expansion by adding new movies, users, and rating data.